



Qelem Meda Technologies

Junior Full Stack Developer Technical Exam

Candidate Information

Name: Addisu Berihun Ademe

Phone: 0976 863616 Email: berihunaddisu9@gmail.com

Exam Duration: 6 hrs.

Total Mark: 100

Instructions to Candidates

- Answer all questions thoroughly.
- Provide clear, complete, and well-structured responses.
- Manage your time effectively to complete every section.
- Follow all instructions carefully and consider the evaluation criteria to maximize your score.

Detail Instructions

1. Duration: 6 hours total (20 Minutes for multiple choice, 5: 40 for practical).
2. Environment: You must complete the test individually on your local machine using Laravel, Vue.js 3 (Composition API), and Tailwind CSS.
3. Tools allowed
 - ✓ Text editor or IDE (VS Code, Sublime, etc.)
 - ✓ Browser (for testing your code locally).
 - ✓ Local database (MySQL, Postgres or SQLite).
 - ✓ Documentation lookup on Laravel, Vue.js, or Tailwind CSS official docs is allowed.
4. Tools NOT allowed
 - ✓ AI-generated code (ChatGPT, GitHub Copilot, Bard, etc.)
 - ✓ Copy-pasting from online forums or GitHub repositories.
 - ✓ Collaboration or communication with others during the exam.
5. Code Requirements
 - ✓ Write clean, well-structured code.
 - ✓ Use meaningful variable and function names.
 - ✓ Organize Laravel code following MVC principles.
 - ✓ Vue components must use Composition API syntax
 - ✓ Tailwind CSS should be used for all UI styling.
6. Working and Submission Format
 - ✓ You are provided with two repositories:
 - Backend (Laravel): <https://github.com/gelemmeda/dev-project-backend.git>
 - Frontend (Vue.js + Tailwind): <https://github.com/gelemmeda/dev-project-front.git>
 - ✓ Clone both repositories and implement your solution.
 - ✓ Ensure clean structure, best coding practices, and proper backend–frontend integration.
 - ✓ Include your name, email, and clear setup/run instructions in each project's README.md file.
 - ✓ Push your completed backend and frontend work to their respective repositories before submission.

Part I: Multiple Choice (10 marks)

1. In Vue.js, what is the correct syntax for binding a variable to an element's attribute?
A. bind-src="image"
B. :src={image}
C. v-src="image"
☒ D. v-bind:src="image"
2. Which lifecycle hook in Vue is called when a component is first created?
A. beforeMount
B. onMounted
☒ C. beforeCreate
D. updatedxx
3. What does php artisan serve do?
A. Runs database migrations
☒ B. Starts a local development server
C. Installs dependencies
D. Creates controllers
4. Which Laravel feature simplifies working with RESTful APIs?
☒ A. Resource controllers
B. Middleware
C. Events
D. Listeners
5. How can you define a one-to-many relationship in Eloquent?
☒ A. return \$this->hasMany(Post::class);
B. return \$this->belongsTo(Post::class);
C. return \$this->hasOne(Post::class);
D. return \$this->belongsToMany(Post::class);

Part II: Practical Questions (90 marks)

Title: Mini Digital Menu App

Develop a mini digital menu app for restaurants, cafés, hotels, and similar service providers using Laravel (backend), Vue.js (frontend), and Tailwind CSS (UI).

Each service provider (restaurant, cafe, or hotel) can register, create simple menu categories and items, and generate a public menu link for customers or accessible through a QR code. Customers browse a public menu link or scan the QR code to view menus and place simple orders.

1. User Registration & Authentication (20 Marks)

Build a registration and login form for service providers.

Requirements

- ✓ Use Laravel Breeze, Sanctum, or Passport for authentication.
- ✓ Registration form fields: name, email, phone, password, business_name, tin.
- ✓ Validate inputs (required fields, unique email and tin).
- ✓ Hash passwords before saving.
- ✓ After registration, simulate sending credentials (display success message with credentials).

Evaluation criteria

- ✓ Correct use of Laravel authentication
- ✓ Proper use of Laravel Model & Controller for registration
- ✓ Secure password hashing
- ✓ Clear UI (Tailwind CSS layout for registration and login)
- ✓ Basic validation (unique email and tin, required fields)

2. Menu Category Management (20 Marks)

Allow authenticated users to manage menu categories.

Example: Breakfast, Lunch, Dinner, Snack, Soft Drink, Alcohol, Hot Drink

Requirements

- ✓ Fields: name, image (optional), description (optional).
- ✓ Implement CRUD (Create, Read, Update, Delete).
- ✓ Display all categories in a Vue.js component.
- ✓ API Routes and Use Axios for API calls.

Evaluation criteria

- ✓ Proper schema & migration for menu categories
- ✓ CRUD functionality implemented
- ✓ Request validation for inputs
- ✓ Proper JSON response
- ✓ Proper API integration (Axios)
- ✓ Dynamic Vue list rendering, Tailwind layout

3. Menu Item Management (20 Marks)

Allow service providers to manage items under each category.

Requirements

- ◆ Fields: item_name, category_id, price, tax_percentage, photo (optional).
- ◆ Automatically calculate final price = $\text{price} + (\text{price} \times \text{tax_percentage} / 100)$.
- ◆ Show items grouped by category.
- ◆ Implement simple search/filter (by item name or category).

Evaluation criteria

- ✓ Correct model relationships (Category → MenuItem)
- ✓ Build form and validation for menu item registrations
- ✓ Computed tax handling
- ✓ Clean UI for menu item registration and display
- ✓ Simple search and filter

4. Customer Menu Viewing (15 Marks)

Generate a public menu page for customers to view menu items.

Requirements

- ❖ Generate a unique URL for each service providers (e.g., /menu/{slug}).
- ❖ Show all categories and items for that service provider.
- ❖ No authentication required.

Evaluation criteria

- ✓ Proper web route handling (Route::get('/menu/{slug}'))
- ✓ Fetch service provider and related items via relationships

5. Database Seeder & Demo Data (15 Marks)

Create seeders for demo data including

Requirements

- ❖ 2-3 demo users
- ❖ Sample categories and menu items

Evaluation criteria

- ✓ Proper use of Laravel seeder classes (DatabaseSeeder, UserSeeder, CategorySeeder, MenuItemSeeder) with logical structure and calling order.
- ✓ Each MenuItem belongs to a valid Category, and each Category is linked to a valid User while creating seeder
- ✓ Seeder runs successfully using php artisan db:seed without errors.